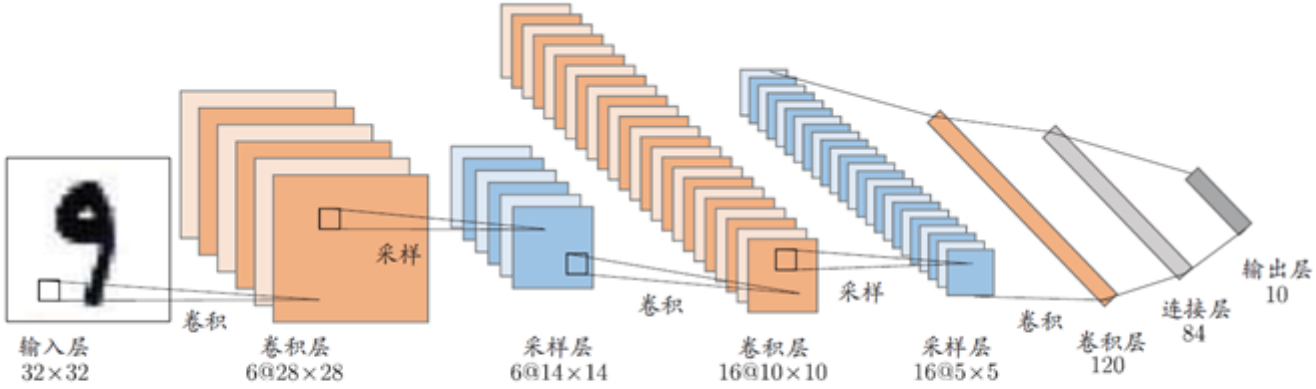
14.1 深度学习发展简史
14.2 多层感知机
14.3 损失函数
14.4 优化算法
14.5 案例：手写数字识别
14.6 深度学习技巧
14.7 卷积神经网络
14.8 案例：图像识别

卷积神经网络

结构：CNN复合多个卷积层和采样层对输入信号进行加工，然后在连接层实现与输出目标之间的映射.



卷积神经网络用于手写数字识别 [LeCun et al., 1998]

权共享

- 一组神经元使用相同的连接权值.
- 权共享策略在卷积神经网络(CNN)[LeCun and Bengio，1995; LeCun et al.，1998]中发挥了重要作用.

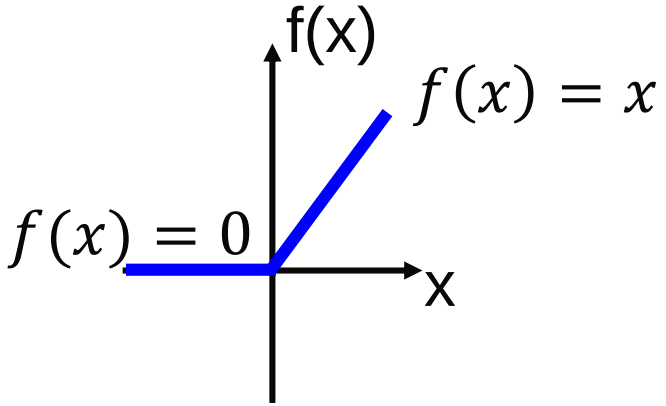
卷积神经网络

- 卷积层：每个卷积层包含多个特征映射，每个特征映射是一个由多个神经元构成的“平面”，通过一种卷积滤波器提取的一种特征
- 采样层：亦称“汇合层”，其作用是基于局部相关性原理进行亚采样，从而在减少数据量的同时保留有用信息
- 连接层：每个神经元被全连接到上一层每个神经元，本质就是传统的神经网络，其目的是通过连接层和输出层的连接完成识别任务

卷积神经网络激活函数

在CNN中通常将 sigmoid 激活函数替换为修正的线性函数

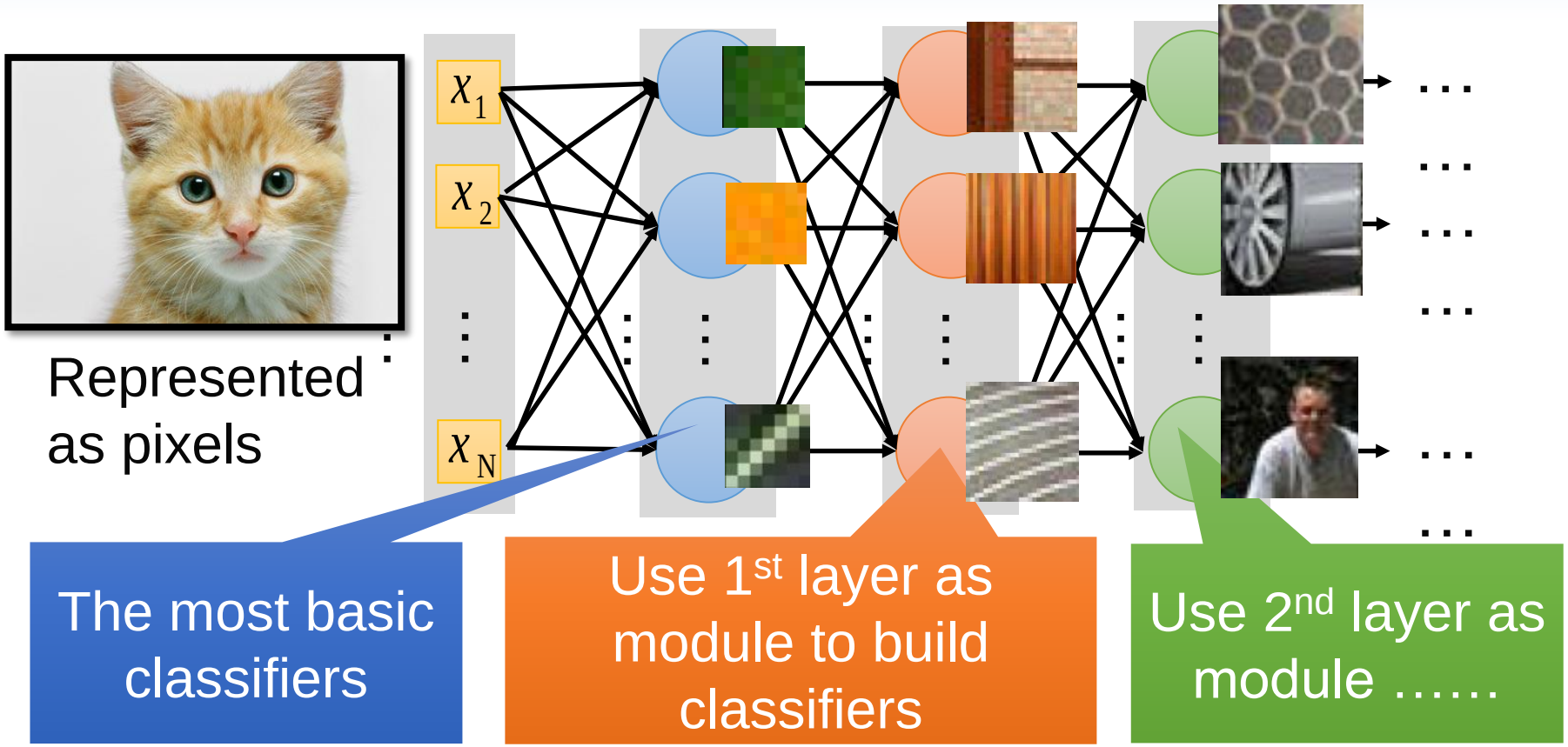
$$f(x) = \max(0, x)$$



卷积神经网络训练

CNN 可以用BP进行训练，但在训练中，无论是卷积层还是采样层，每一组神经元都是用相同的连接权，从而大幅减少了需要训练的参数数目

Why CNN for Image?



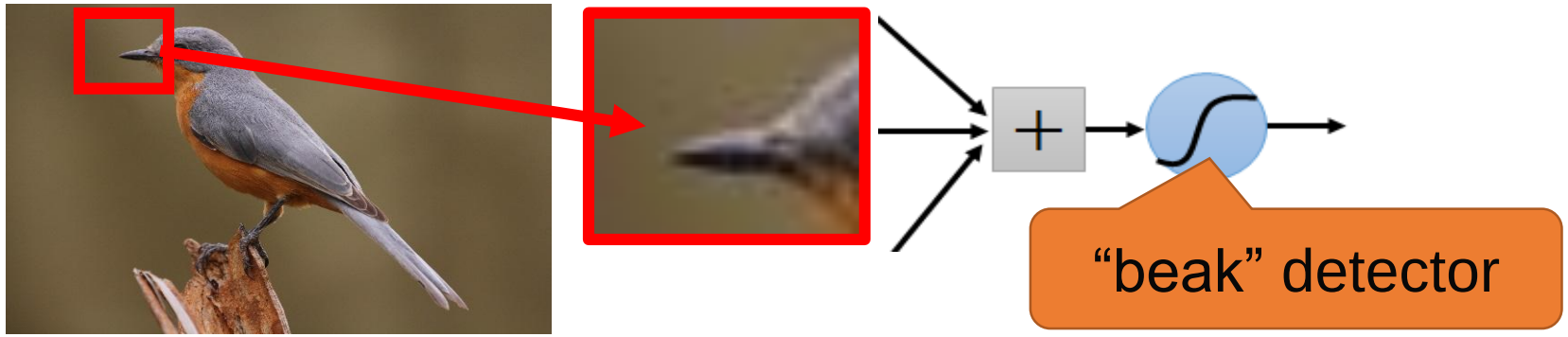
Can the network be simplified by considering the properties of images?

Why CNN for Image

Some patterns are much smaller than the whole image

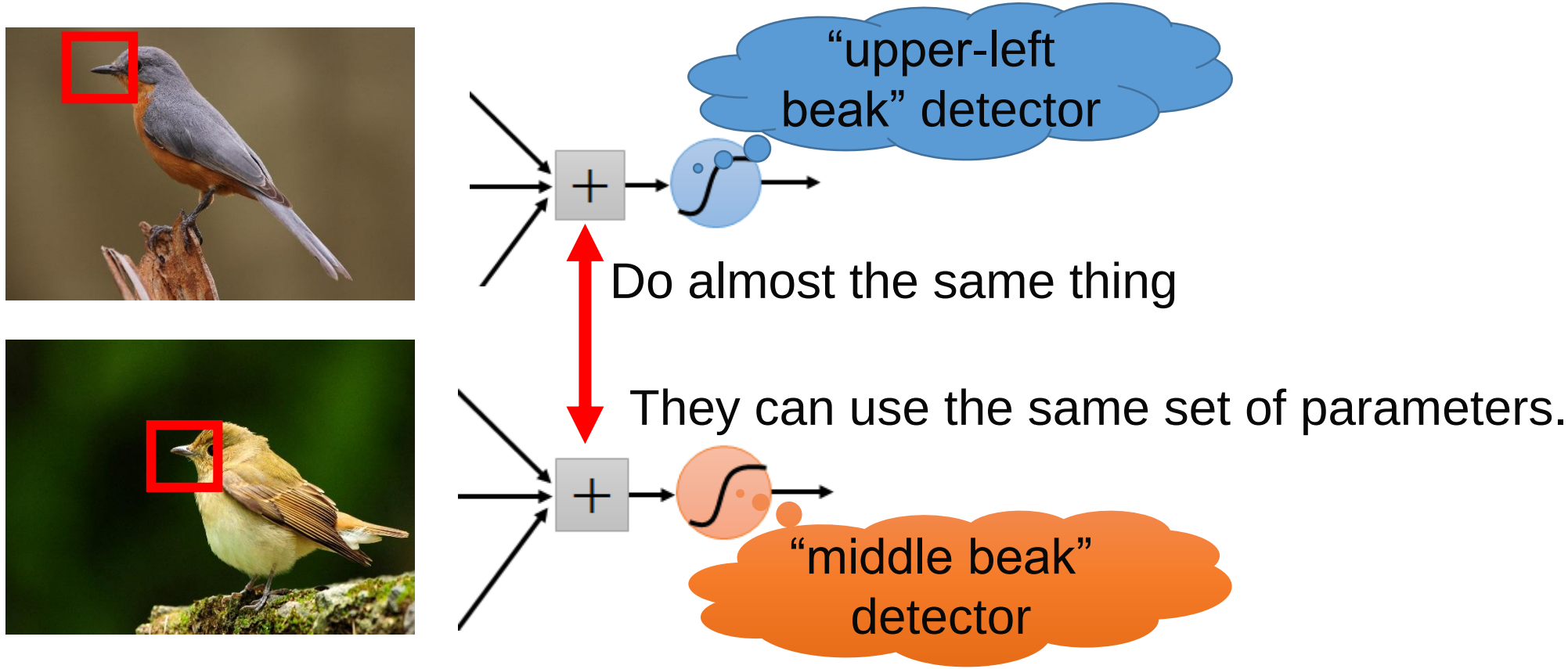
A neuron does not have to see the whole image to discover the pattern.

Connecting to small region with less parameters



Why CNN for Image

The same patterns appear in different regions.



Why CNN for Image

Subsampling the pixels will not change the object

bird

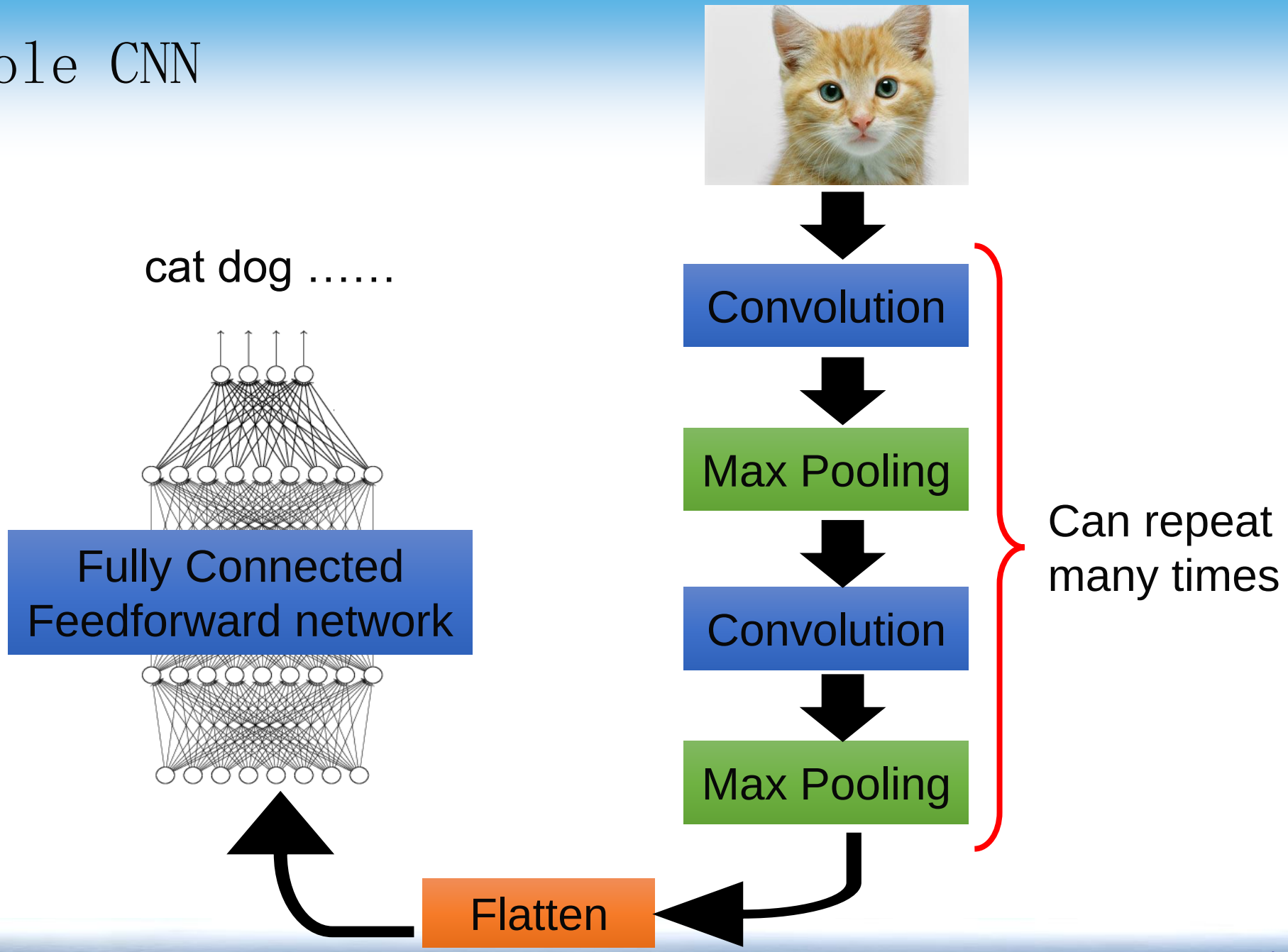
subsampling

bird

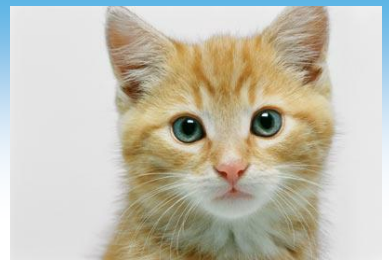
We can subsample the pixels to make image smaller

→ Less parameters for the network to process the image

The whole CNN



The whole CNN



Property 1

- Some patterns are much smaller than the whole image

Property 2

- The same patterns appear in different regions.

Property 3

- Subsampling the pixels will not change the object

Convolution

Max Pooling

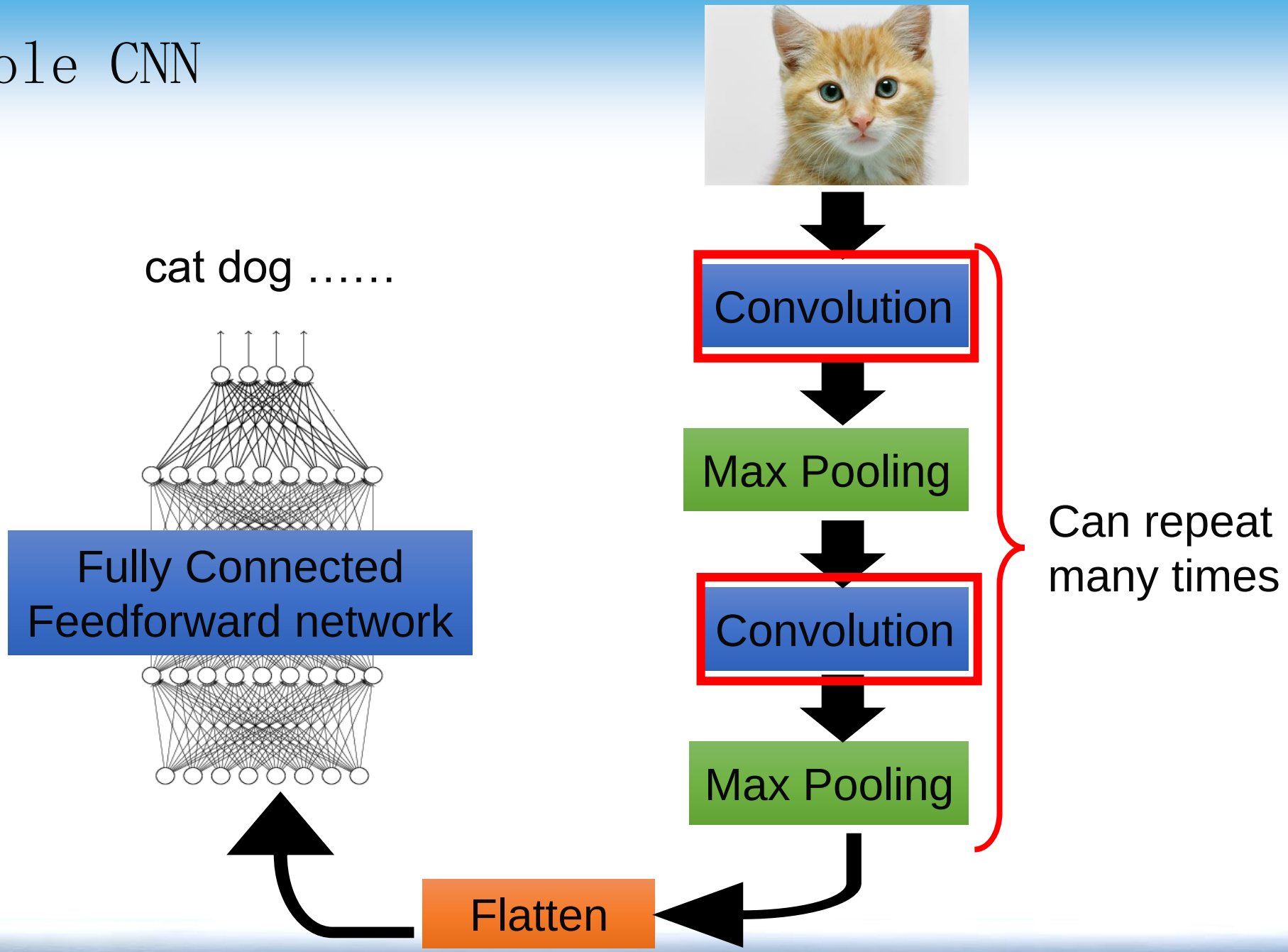
Convolution

Max Pooling

Can repeat many times

Flatten

The whole CNN



CNN - Convolution

Those are the network parameters to be learned.

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

Matrix

-1	1	-1
-1	1	-1
-1	1	-1

Filter 2

Matrix

⋮ ⋮

Property 1

Each filter detects a small pattern (3 x 3).

CNN - Convolution

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

stride=1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

3 -1

CNN - Convolution

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

If stride=2

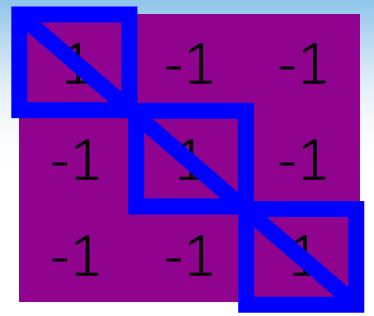
1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

3 -3

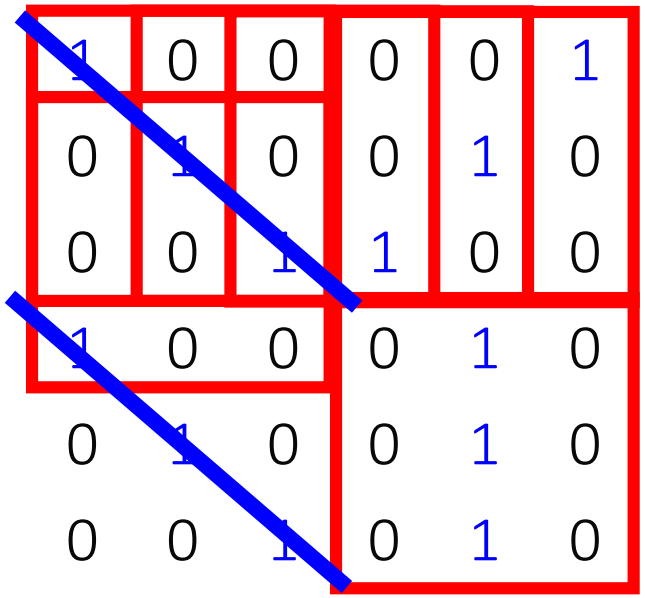
We set stride=1 below

CNN - Convolution

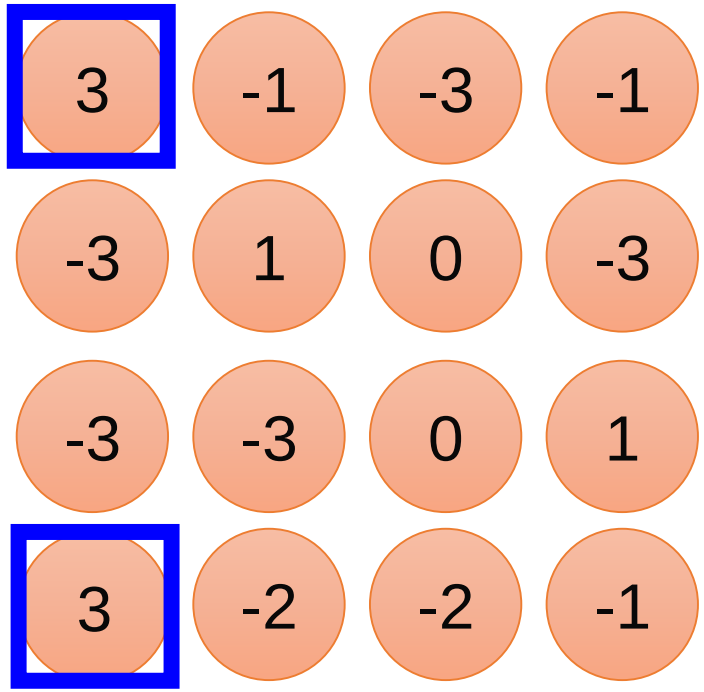


Filter 1

stride=1



6 x 6 image



Property 2

CNN - Convolution

-1	1	-1
-1	1	-1
-1	1	-1

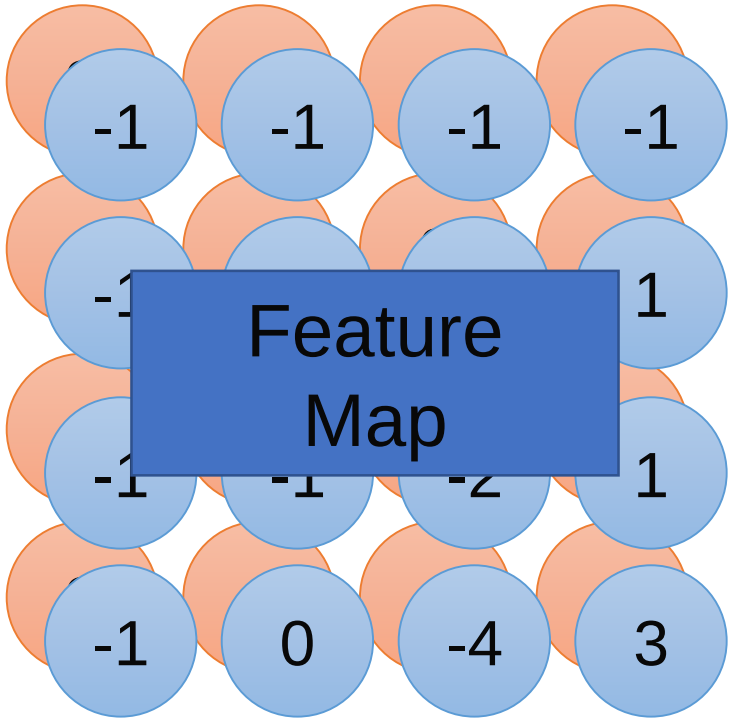
Filter 2

stride=1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

Do the same process for every filter



4 x 4 image

CNN - Zero Padding

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

0	0	0				
0	1	0	0	0	0	1
0	0	1	0	0	1	0
	0	0	1	1	0	0
	1	0	0	0	1	0
	0	1	0	0	1	0
	0	0	1	0	1	0
					0	0
					0	0

6 x 6 image

You will get another 6 x 6 images in this way

Zero padding

CNN - Colorful image

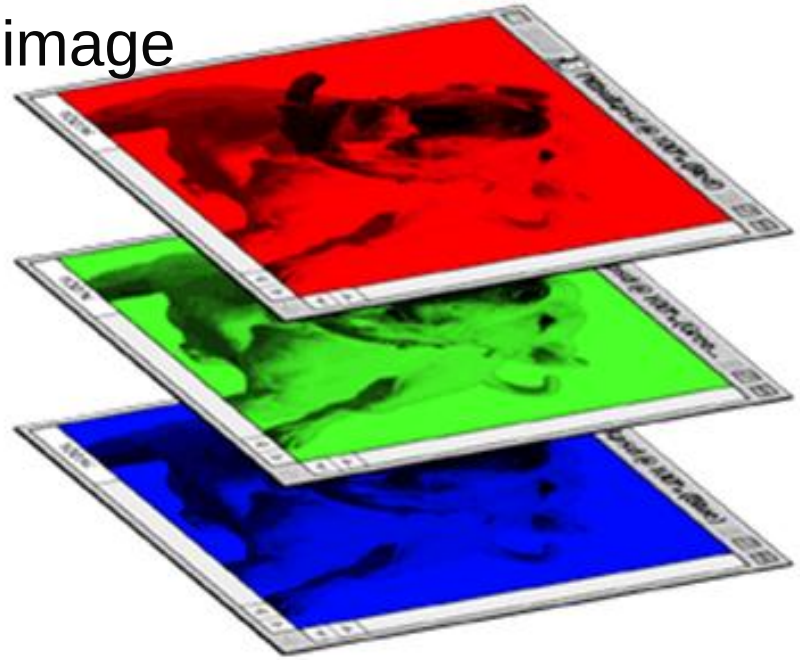
1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

-1	1	-1
-1	1	-1
-1	1	-1

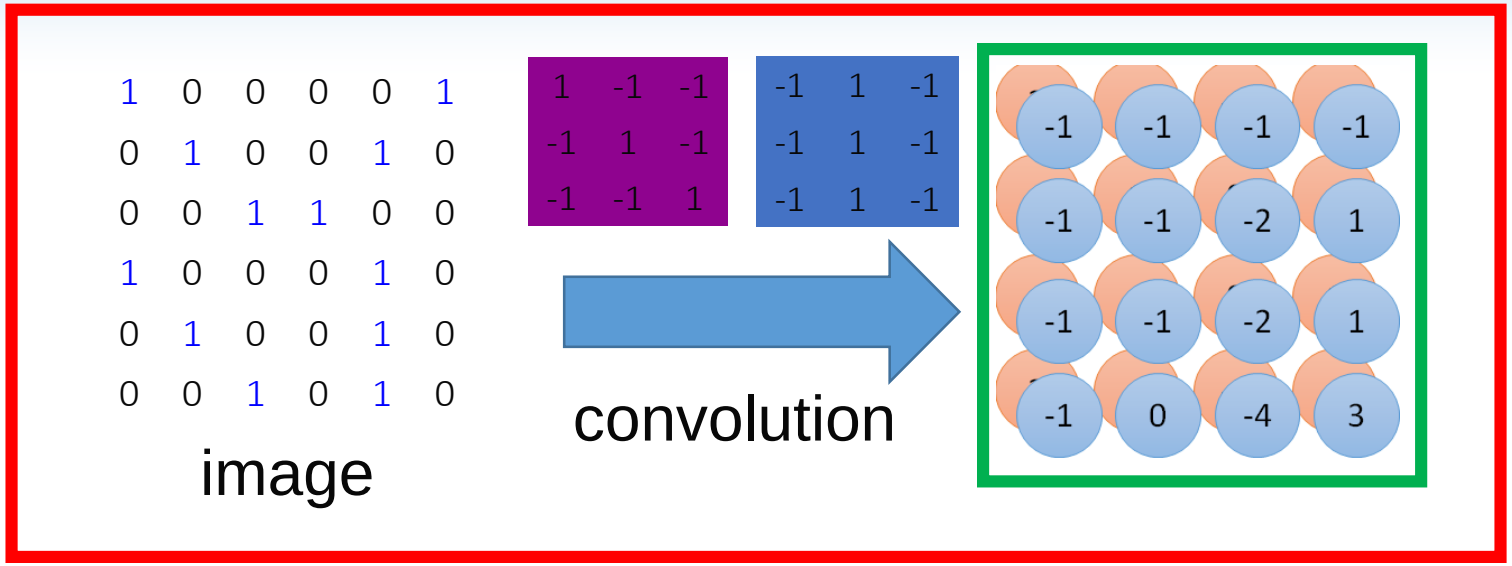
Filter 2

Colorful image

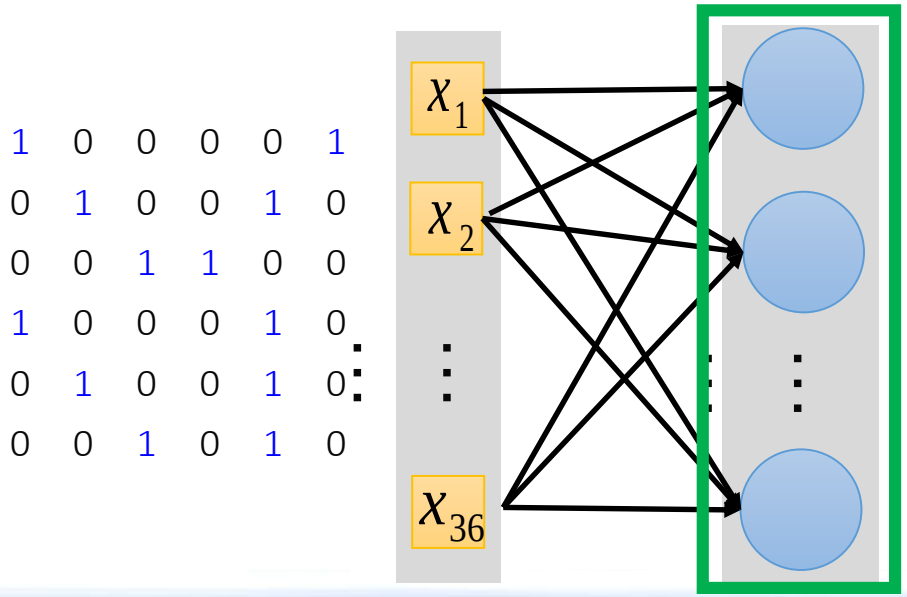


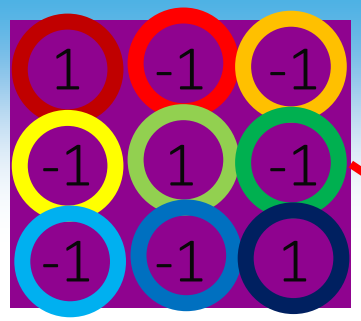
1	0	0	0	0	1		
0	1	0	0	0	0	1	
0	0	1	0	0	0	0	1
1	0	0	1	0	0	1	0
0	1	0	0	1	1	0	0
0	0	1	1	0	0	1	0
0	0	0	1	0	0	1	0
	0	0	0	1	0	1	0

Convolution v.s. Fully Connected



Fully connected



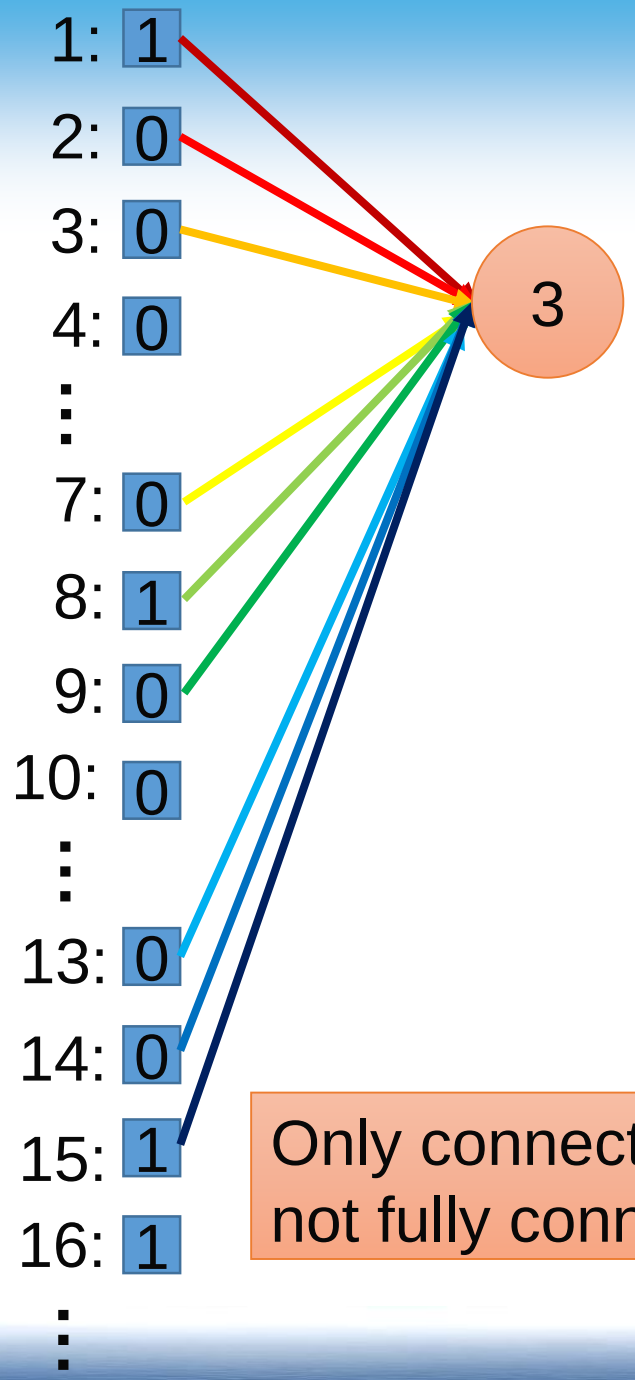


Filter 1

1	0	0	0	0	
0	1	0	0	1	
0	0	1	1	0	
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

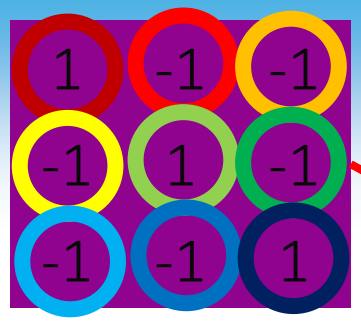
6 x 6 image

3	-1	-3	-1
-3	1	0	-3
-3	-3	0	1
3	-2	-2	-1



Less parameters!

Only connect to 9 input, not fully connected

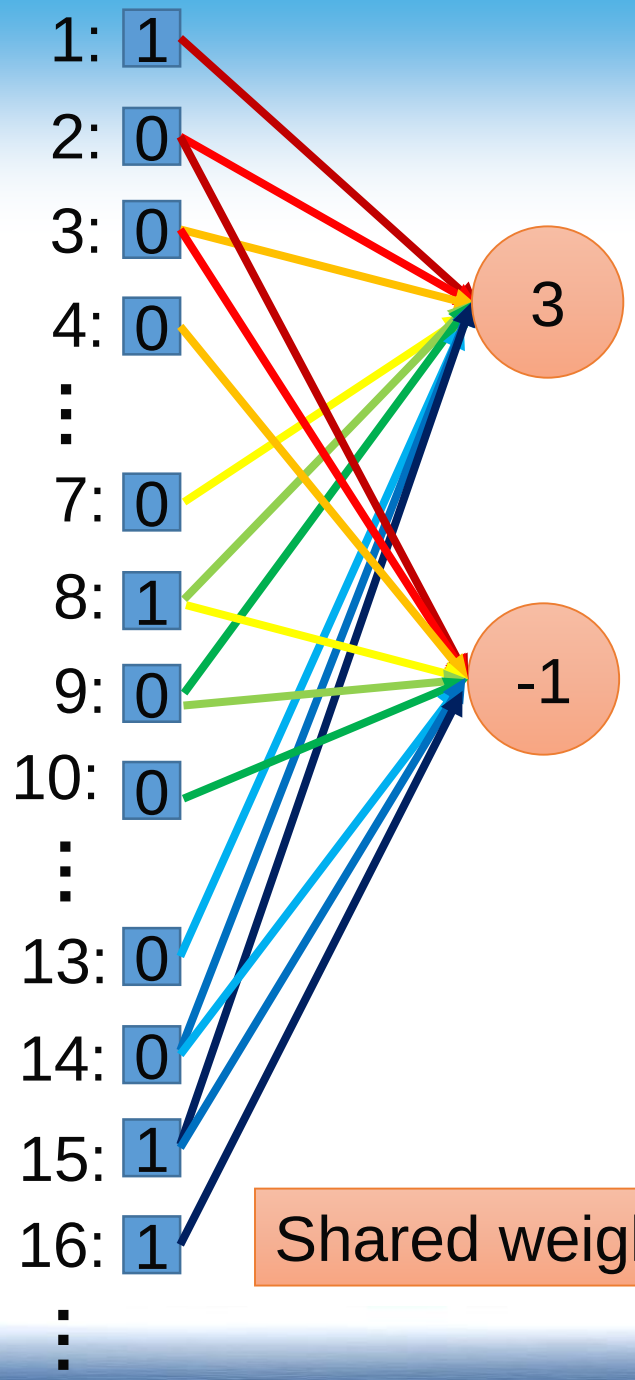


Filter 1

1	0	0	0	0	0
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

3	-1	-3	-1
-3	1	0	-3
-3	-3	0	1
3	-2	-2	-1

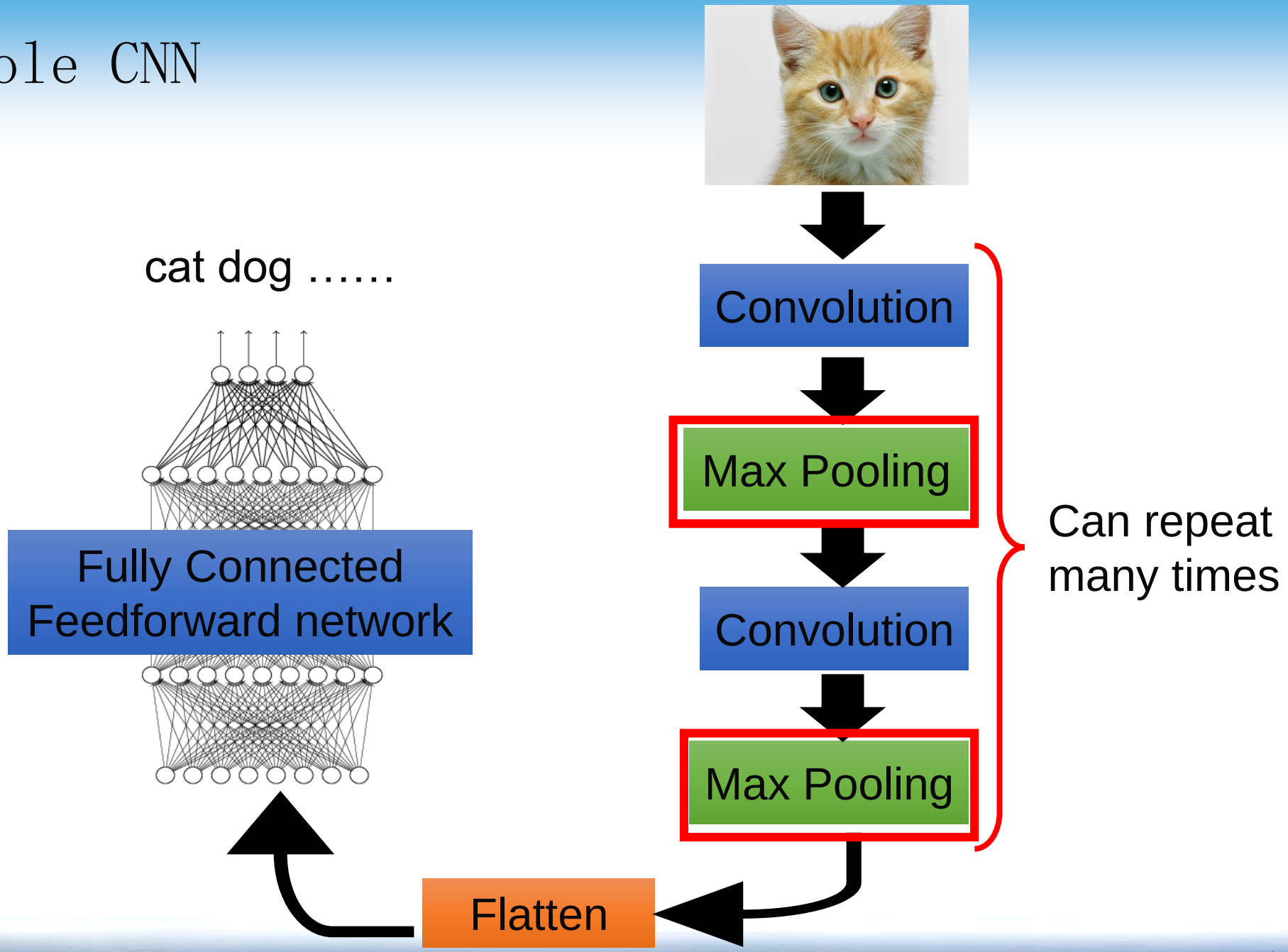


Less parameters!

Even less parameters!

Shared weights

The whole CNN



CNN - Max Pooling

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

-1	1	-1
-1	1	-1
-1	1	-1

Filter 2

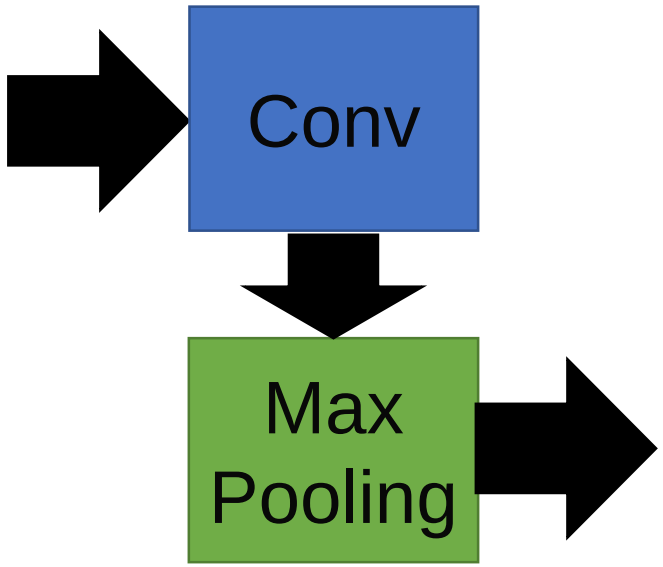
3	-1	-3	-1
-3	1	0	-3
-3	-3	0	1
3	-2	-2	-1

-1	-1	-1	-1
-1	-1	-2	1
-1	-1	-2	1
-1	0	-4	3

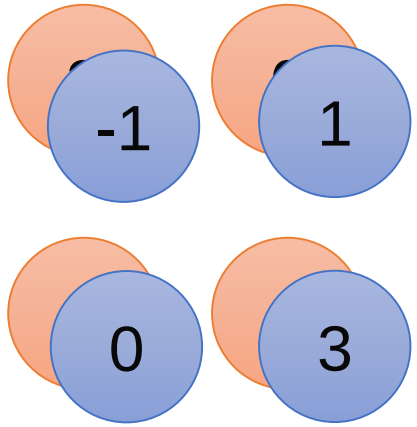
CNN - Max Pooling

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image



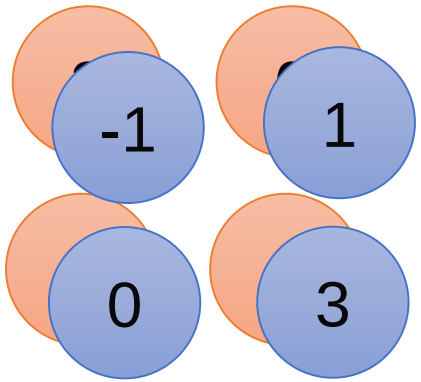
New image but smaller



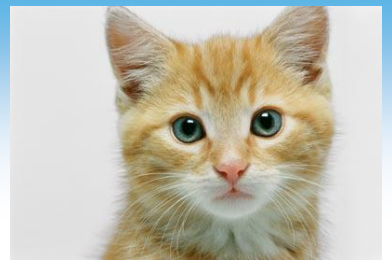
2 x 2 image

Each filter is a channel

The whole CNN



A new image



Convolution

Max Pooling

Convolution

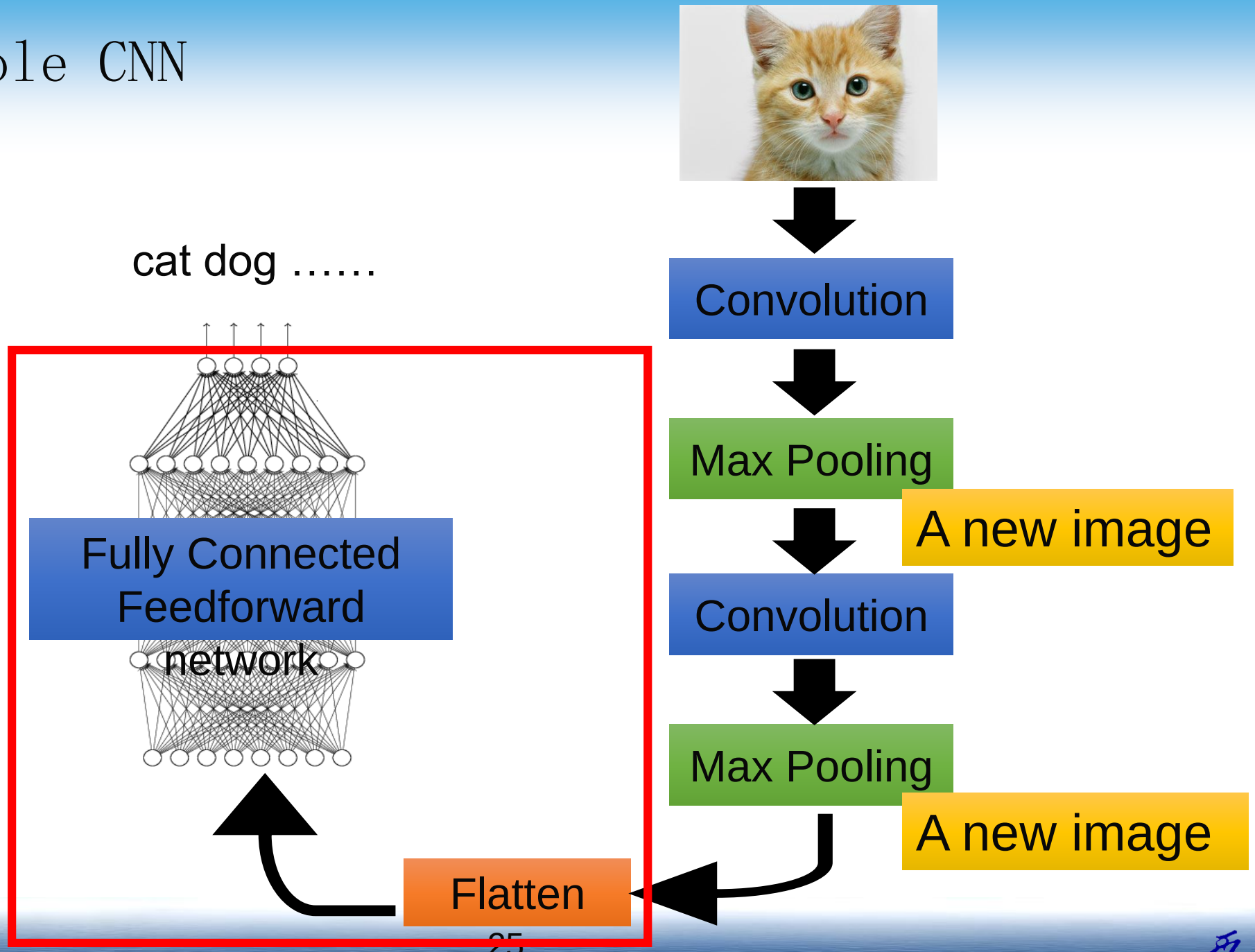
Max Pooling

Can repeat many times

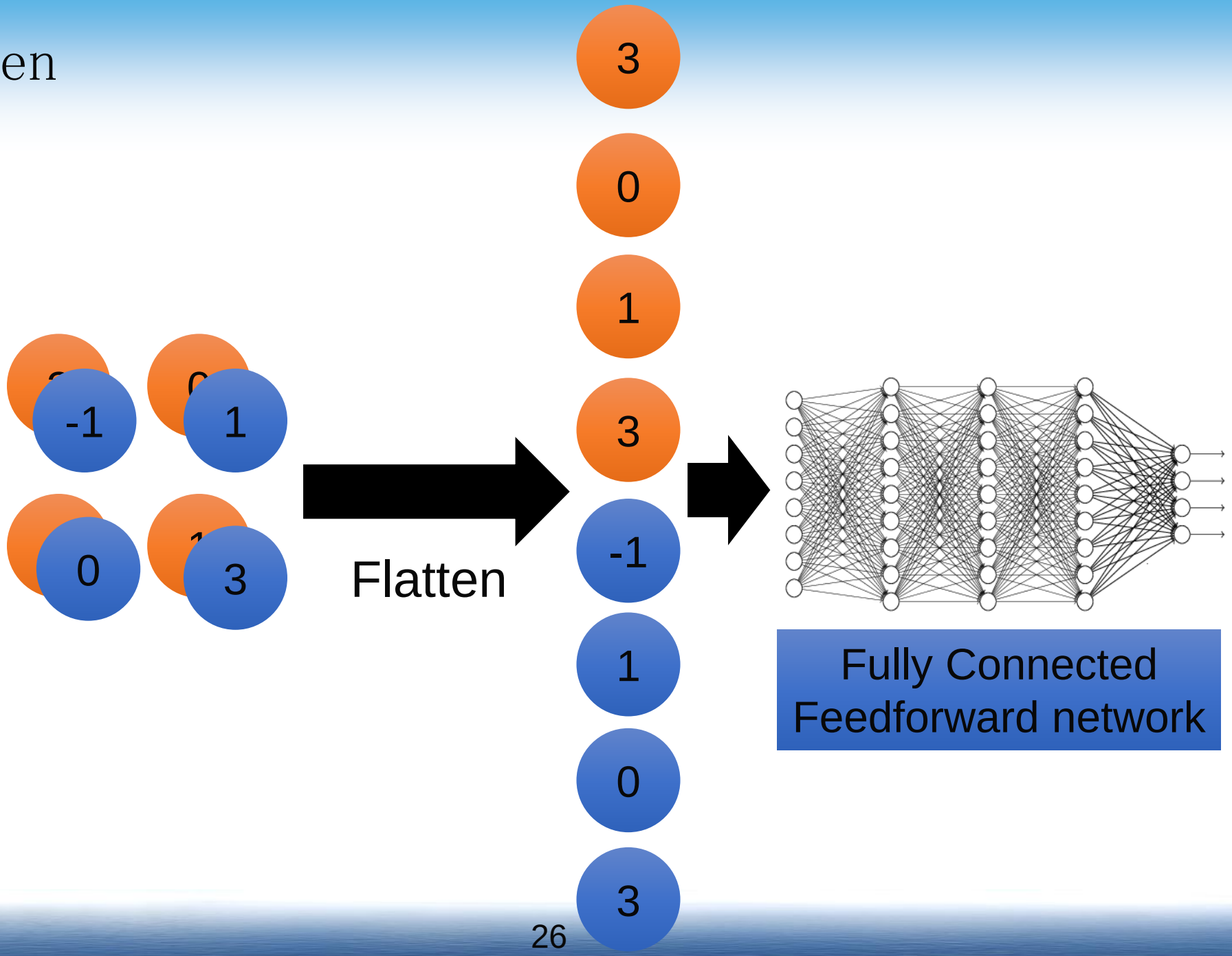
Smaller than the original image

The number of the channel is the number of filters

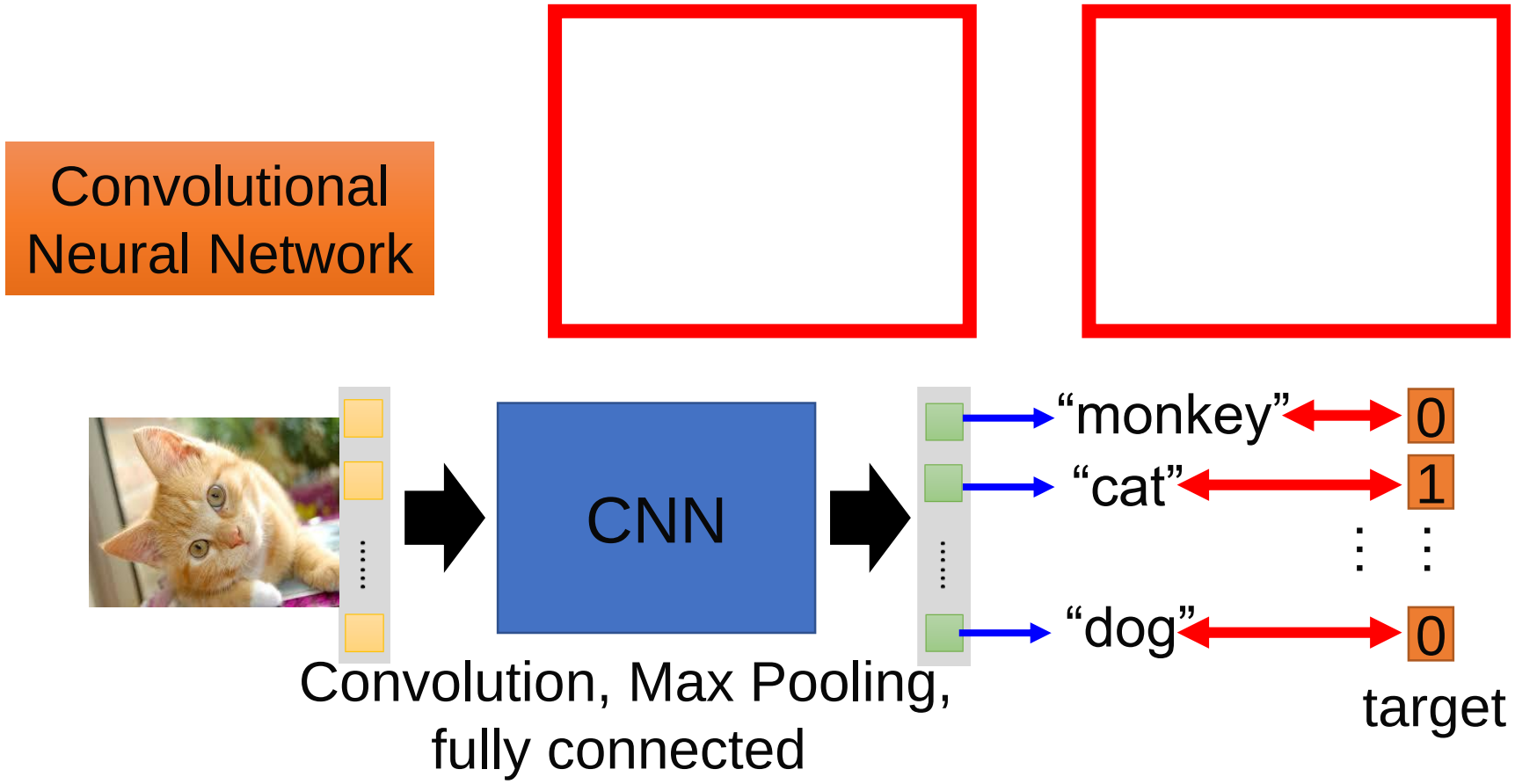
The whole CNN



Flatten


-1
1
0
3
Flatten
3
0
1
3
-1
1
0
3
Fully Connected
Feedforward network

Convolutional Neural Network

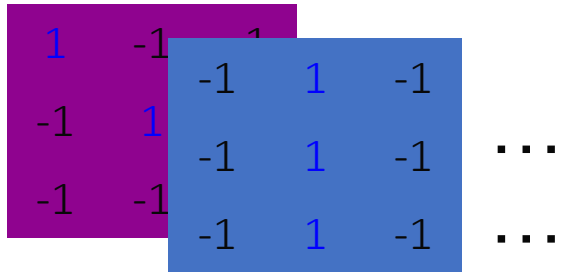


Learning: Nothing special, just gradient descent ……

CNN in Keras

Only modified the ***network structure*** and ***input format (vector -> 3-D tensor)***

```
model2.add( Convolution2D( 25,3,3,
            input_shape=(1,28,28) ) )
```

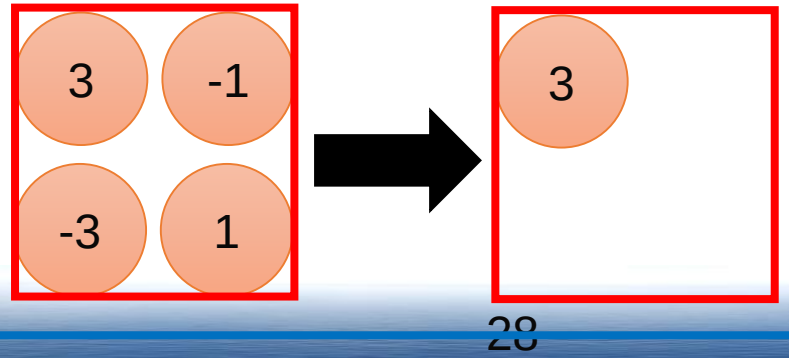


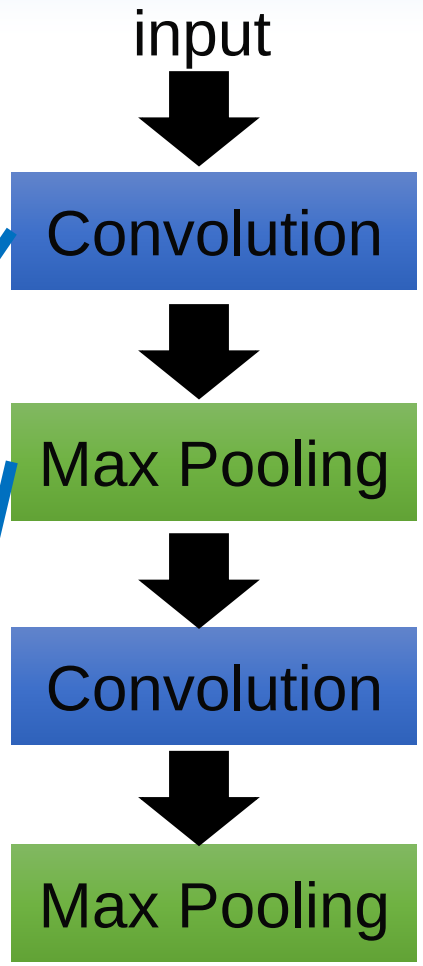
There are 25 3x3 filters.

Input_shape = (1 , 28 , 28)

1: black/weight, 3: RGB 28 x 28 pixels

```
model2.add(MaxPooling2D((2,2)))
```





CNN in Keras

Only modified the ***network structure*** and ***input format (vector -> 3-D tensor)***

input

1 x 28 x 28

```
model2.add( Convolution2D( 25,3,3,
            input_shape=(1,28,28) ) )
```

Convolution

How many parameters for each filter?

9

25 x 26 x 26

```
model2.add(MaxPooling2D((2,2)))
```

Max Pooling

25 x 13 x 13

```
model2.add(Convolution2D(50,3,3))
```

Convolution

How many parameters for each filter?

225

50 x 11 x 11

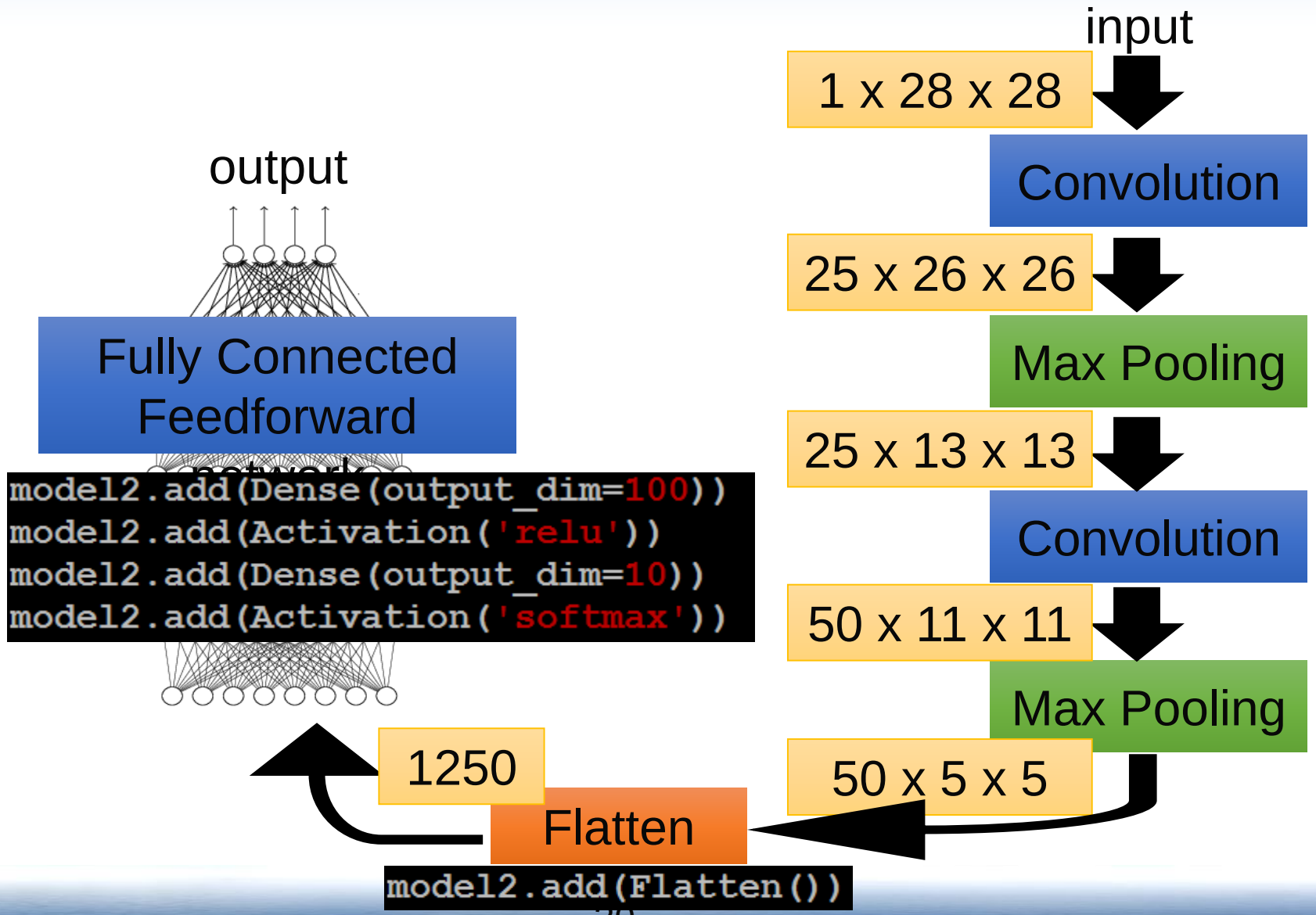
```
model2.add(MaxPooling2D((2,2)))
```

Max Pooling

50 x 5 x 5

CNN in Keras

Only modified the ***network structure*** and ***input format (vector -> 3-D tensor)***



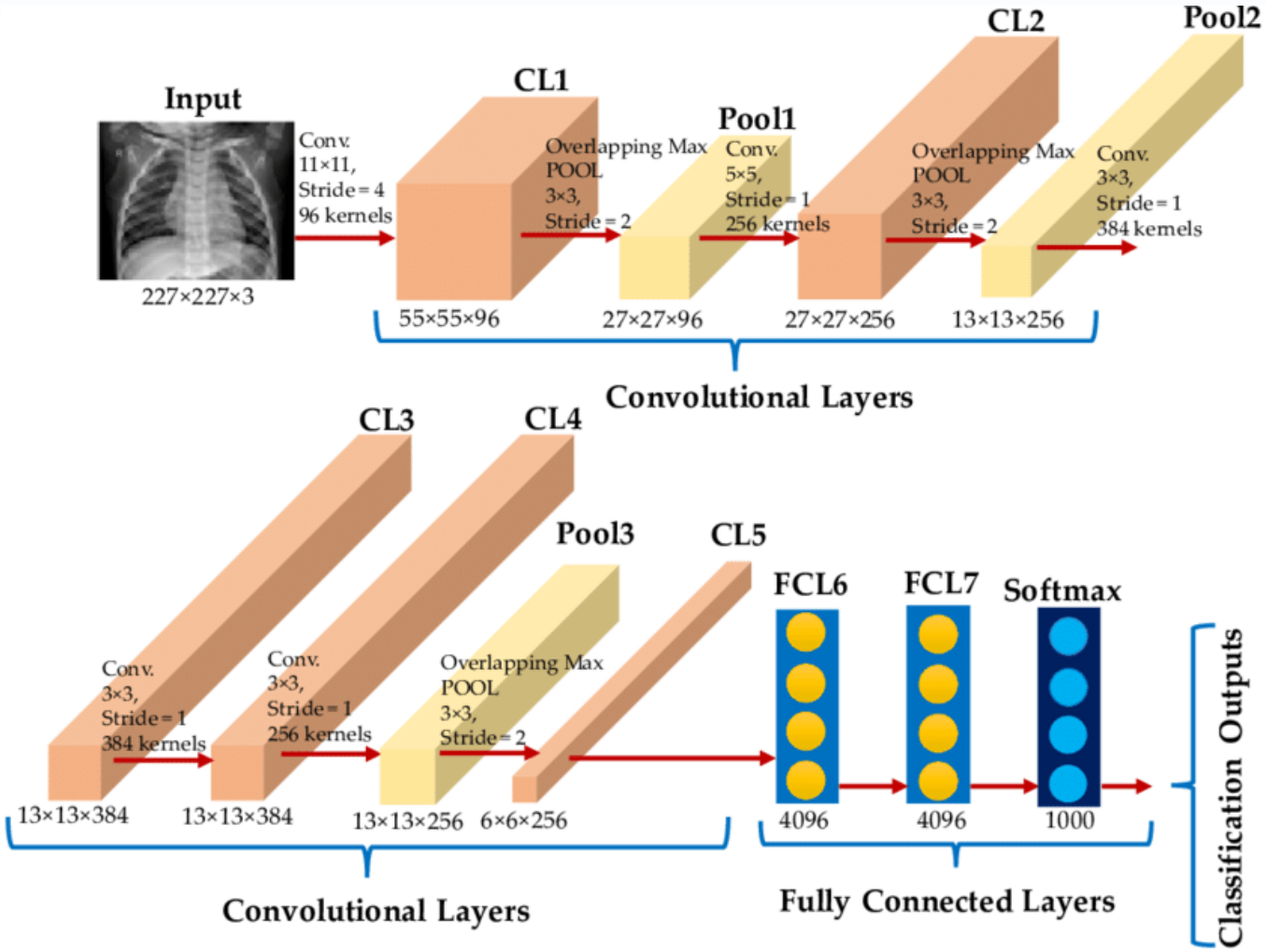

```
model2.add(Dense(output_dim=100))
model2.add(Activation('relu'))
model2.add(Dense(output_dim=10))
model2.add(Activation('softmax'))
```

```
model2.add(Flatten())
```

AlexNet

AlexNet 是首个真正意义上的深度卷积神经网络

- 首次使用了ReLU非线性激活函数
- 局部响应归一化（LRN）
- 数据增强
- Dropout 技术
- 加入动量的小批量梯度下降算法
- 使用多GPU进行训练

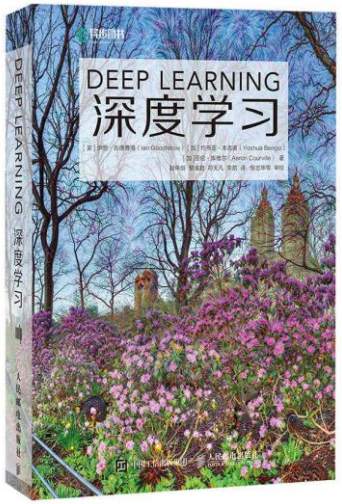


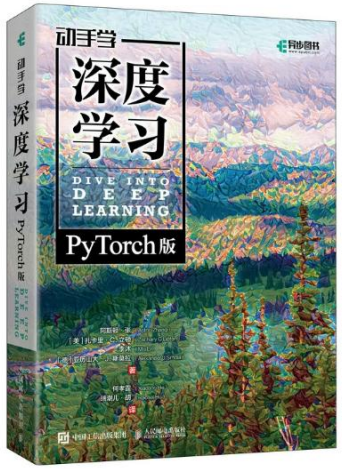
	参数个数		FLOP	
	AlexNet	LeNet	AlexNet	LeNet
Conv1	35K	150	101M	1.2M
Conv2	614K	2.4K	415M	2.4M
Conv3-5	3M		445M	
Dense1	26M	0.48M	26M	0.48M
Dense2	16M	0.1M	16M	0.1M
Total	46M	0.6M	1G	4M

深度学习框架

- TensorFlow
- PyTorch
- Keras
- PaddlePaddle

scikit-neuralnetwork





14.1 深度学习发展简史
14.2 多层感知机
14.3 损失函数
14.4 优化算法
14.5 案例：手写数字识别
14.6 深度学习技巧
14.7 卷积神经网络
14.8 案例：图像识别